Table of Contents

跨境电商选品调研 Agent 技术设计说明书

> 本文档为《跨境电商选品调研 Agent 需求规格说明书》（V1.0）的配套技术设计文档，基于现有项目代码反推生成，用于描述系统的技术架构、模块设计、接口、数据存储、部署与测试方案。

---

1. 文档概述

1.1 文档基本信息

项目	内容
项目名称	跨境电商选品调研 Agent（Product Research Agent）
文档名称	跨境电商选品调研 Agent 技术设计说明书
文档版本	V1.0
文档状态	草稿
编制日期	2026-10-22
编制方式	基于现有代码 + 需求规格说明书反推生成
对应需求文档	《跨境电商选品调研 Agent 需求规格说明书》V1.0

1.2 编写目的

本文档用于定义跨境电商选品调研 Agent 系统的：

- 系统总体架构与设计原则；
- 技术选型与技术栈；
- 各功能模块的详细设计；
- 数据存储方案（含内存缓存设计）；
- 接口设计（LLM 接口、工具接口、爬虫接口、流式事件协议）；
- 核心业务流程；
- 安全、性能、异常、日志设计；
- 部署与测试方案；
- 风险与扩展规划。

1.3 文档范围

本文档覆盖系统的技术实现方案，与需求文档的分工如下：

内容	需求文档	本文档
用户需求 / 业务流程 / 验收标准	✓	—
架构设计 / 模块设计	—	✓
数据库 / 缓存设计	—	✓
接口实现 / 算法设计	—	✓
技术选型 / 爬虫实现	—	✓
部署 / 测试 / 安全方案	—	✓

1.4 读者对象

读者	关注点
后端开发工程师	模块设计、接口设计、业务流程
算法/LLM 工程师	意图分类、趋势判定、评分模型、Prompt 设计
测试工程师	测试方案、验收标准
运维工程师	部署方案、日志、异常处理
架构师	总体架构、技术选型、风险与扩展

1.5 术语表

术语	说明
Pipeline	串行的多阶段分析流程（9 步）
Tool	可独立执行的业务分析单元（继承 BaseTool）
Crawler	数据爬取器（继承 BaseCrawler）
BSR	Amazon Best Sellers Rank，畅销榜排名
FBA	Fulfillment by Amazon，亚马逊物流
ASIN	Amazon Standard Identification Number，商品唯一标识
Listing	Amazon 商品详情页
MOQ	Minimum Order Quantity，最小起订量
LLM	Large Language Model，大语言模型
TTL	Time To Live，缓存存活时间

---

2. 项目背景

2.1 业务背景

跨境电商卖家在选品决策过程中普遍面临以下痛点：

1. **人工处理效率低**——从市场调研到选品决策需在 Amazon、Google Trends、1688 等多个平台间反复切换；

2. **信息获取困难**——市场容量、价格带、竞品销量、评论痛点等关键数据分散；
3. **数据分散**——搜索热度、BSR、FBA 费用、供应商报价缺乏统一整合视图；
4. **决策维度复杂**——选品需同时兼顾需求、趋势、竞争、利润、供应链五个维度；
5. **缺少智能辅助**——缺乏基于 AI 的自动分析与综合评分能力。

2.2 项目定位

本系统是一款面向跨境电商卖家选品决策场景的**智能辅助系统**，输入品类关键词后自动完成从市场分析到选品决策的全流程调研，输出结构化选品调研报告与决策建议。

2.3 建设目标

编号	目标
G-01	输入品类关键词，自动完成完整选品调研流程
G-02	提供市场、趋势、竞品、评论、关键词、利润、供应商等多维分析能力
G-03	将多平台人工调研压缩为单次自动分析
G-04	通过结果缓存降低重复数据获取成本
G-05	通过五维加权评分提升选品决策客观性

2.4 当前版本范围边界

本版本**不包含**（需求文档第 6.2 节明确）：

- 用户账号体系与登录鉴权；
- 持久化数据存储（数据库）；
- 多租户隔离；
- 自动下单/采购执行；
- 广告投放管理；
- 定时任务与批量调研。

---

3. 系统总体设计

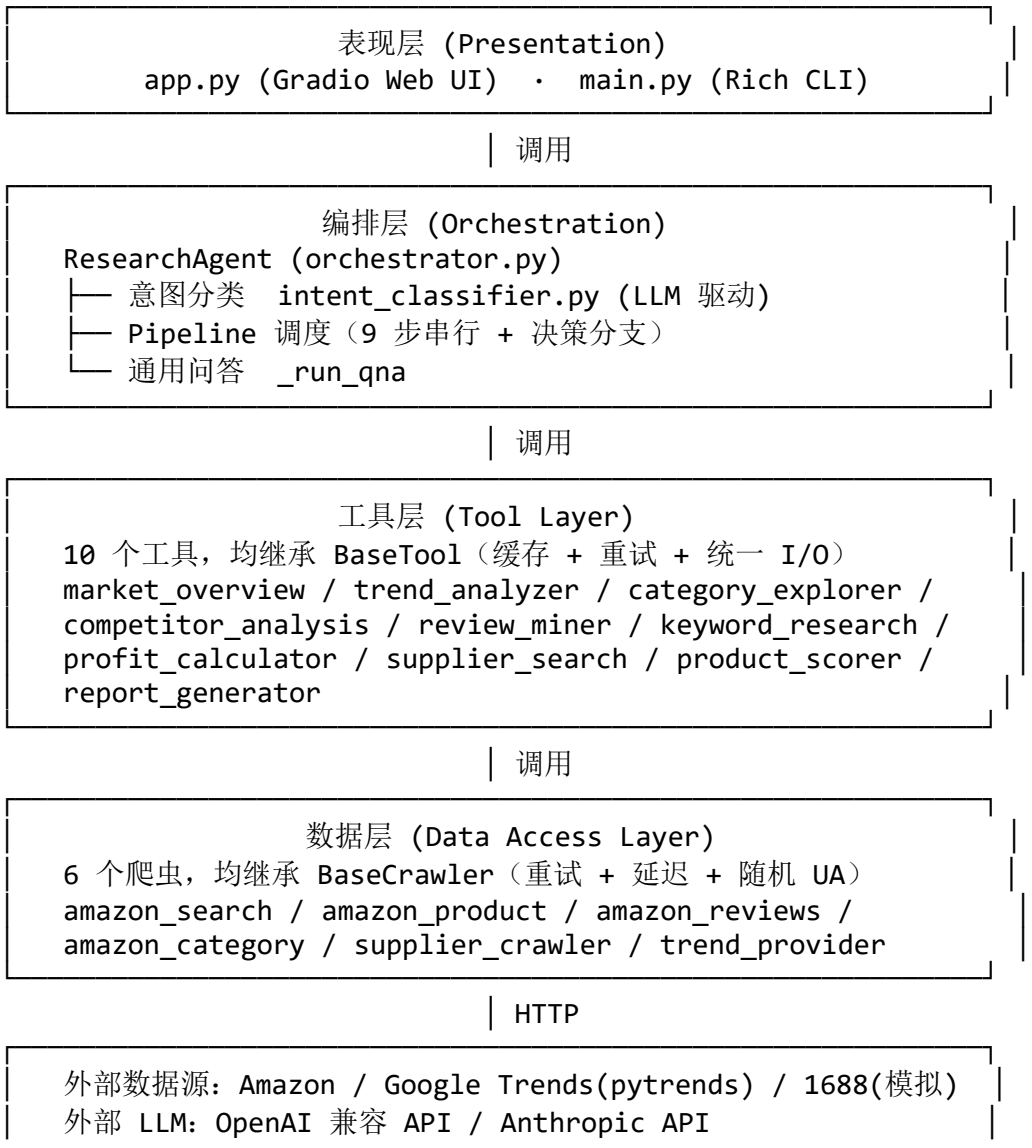
3.1 设计原则

原则	说明	落地体现
分层解耦	表现层 / 编排层 / 工具层 / 数据层职责分离	`app.py→orchestrator.py→tools/→crawlers/`
统一抽象	同类组件共享基类，降低新增成本	`BaseTool`、`BaseCrawler`、`TrendProvider`

单一职责	每个工具只做一件事	10 个独立 Tool 对应 10 个分析维度
失败降级	局部失败不阻断整体流程	单工具失败不中断 Pipeline
数据诚实	不编造结果，明确标注置信度与来源	每个 `ToolOutput` 携带 `source/confidence/evidence`
配置外置	敏感信息与可变参数走环境变量	`.env` + `Settings`

3.2 总体架构

系统采用**分层架构 + Agent 编排**的组合式架构：分层是代码组织方式，Agent 编排（意图路由 + 串行 Pipeline）是运行时行为。



```
└──────────────────────────────────────────────────────────────┘
```

横向贯穿：

```
models/schemas.py    ── Pydantic 数据模型（DTO）
utils/               ── config / cache / helpers
agent/prompt_templates.py ── 所有 LLM Prompt 模板
```

3.3 架构模式识别

模式	应用位置
分层架构	表现层 → 编排层 → 工具层 → 数据层，自上而下单向依赖
意图路由（Router）	`classify_intent` 将自然语言分发到三类处理路径
串行 Pipeline	`_PIPELINE_STAGES` 固定顺序执行，带运行时决策分支
模板方法	`BaseTool.run()` / `BaseCrawler.request()` 定义骨架，子类实现 `execute()`/`_parse()`
策略模式	LLM Provider 切换（`ChatOpenAI` vs `ChatAnthropic`）
注册表模式	`_TOOLS` 全局字典维护工具名→实例映射
观察者/流式	`stream_run` 产出事件，供 UI 渲染进度条

3.4 运行时视图

系统为**单进程单体应用**，支持两种交互模式：

- **CLI 模式**（`main.py`）：同步调用 `agent.run()`，Rich 渲染 Markdown 结果；
- **Web 模式**（`app.py`）：异步调用 `agent.stream_run()` 生成器，逐事件渲染 Pipeline 步骤条与 Plotly 图表。

---

4. 技术架构

4.1 技术栈清单

类别	技术	版本约束	用途
语言	Python 3	3.10+	主开发语言
Agent 框架	langchain / langchain-openai / langchain-anthropic	≥0.3.0	LLM 客户端封装
	langgraph	≥0.2.0	**声明但未实际使用**（见 4.4）
数据模型	pydantic	≥2.0	全系统数据模型与配置
HTTP 客	httpx	≥0.27.0	爬虫 HTTP 请求

户端			
HTML 解析	beautifulsoup4 / lxml	≥4.12 / ≥5.0	解析 Amazon 页面
浏览器自动化	playwright	≥1.40	**声明但未实际使用**
趋势数据	pytrends	≥4.9.0	Google Trends 数据
数值计算	numpy	≥1.24	趋势数据插值
配置	python-dotenv	≥1.0	.env 环境变量加载
CLI	rich	≥13.0	终端美化输出
Web UI	gradio	≥5.0	Web 交互界面
可视化	plotly	≥5.18	图表渲染

4.2 技术选型说明

决策点	选型	理由
LLM 接入	langchain-openai / langchain-anthropic	统一封装多种 OpenAI 兼容 API 与 Anthropic API，支持 DeepSeek、硅基流动等
数据模型	Pydantic v2	类型安全、序列化便捷、与 LLM 结构化输出契合
爬虫	httpx + BeautifulSoup	轻量同步抓取，满足中等规模页面解析
Web UI	Gradio	快速搭建对话式 AI 界面，原生支持流式与图表
缓存	自研内存缓存	无外部依赖，满足单进程场景

4.3 目录结构

```
research_agent/
├── app.py                      # Gradio Web UI 入口
├── main.py                     # CLI 入口 (Rich 终端)
├── requirements.txt            # 依赖清单
├── .env.example                # 环境变量样例
├── agent/
│   ├── orchestrator.py         # 核心：意图路由 + Pipeline 调度
│   ├── intent_classifier.py    # LLM 意图分类器
│   ├── prompt_templates.py     # 全部 Prompt 模板
│   └── state.py                # AgentState 定义
├── tools/
│   ├── base.py                 # BaseTool 抽象基类（缓存+重试）
│   ├── market_overview.py      # 市场概览
│   ├── trend_analyzer.py       # 趋势分析
│   ├── competitor_analysis.py  # 竞品深度分析
│   ├── profit_calculator.py    # 利润测算
```

```
│   ├── keyword_research.py          # 关键词研究
│   ├── review_miner.py              # 评论 NLP 分析
│   ├── supplier_search.py           # 供应商寻源
│   ├── category_explorer.py         # 类目穿透
│   ├── product_scorer.py            # 选品评分卡
│   └── report_generator.py          # LLM 报告生成
├── crawlers/
│   ├── base.py                      # BaseCrawler 基类
│   ├── amazon_search.py             # Amazon 搜索/建议/趋势
│   ├── amazon_product.py            # Amazon 商品详情
│   ├── amazon_reviews.py            # Amazon 评论
│   ├── amazon_category.py           # Amazon 类目（备用）
│   ├── supplier_crawler.py          # 1688 供应商（模拟）
│   └── trend_provider.py            # Google Trends 提供器
├── frontend/
│   ├── pipeline_progress.py         # Pipeline 步骤条 HTML
│   ├── visualizations.py            # Plotly 图表
│   └── utils.py                     # 前端辅助函数
├── models/
│   └── schemas.py                   # Pydantic 数据模型
└── utils/
    ├── config.py                    # 配置管理
    ├── cache.py                     # 内存缓存
    └── helpers.py                   # 工具函数
```

4.4 关键技术观察（如实说明）

1. **langgraph 未使用**：`requirements.txt` 声明了 `langgraph>=0.2.0`，但全代码库无任何 `import langgraph`；Pipeline 编排实为手写 `for` 循环 + generator 实现。
2. **playwright 未使用**：同样仅声明于依赖，实际爬虫全部使用 `httpx` + `BeautifulSoup`。
3. **供应商/部分数据为模拟**：`supplier_crawler.py` 完全返回随机模拟数据；`amazon_reviews.py` 在抓取失败时返回 `_mock_reviews` 模拟评论；`amazon_product.py` 的销量基于 BSR/评论数**随机估算**。
4. **类目树为硬编码**：`category_explorer.py` 使用内置的 `_AMAZON_CATEGORY_TREE` 字典，`amazon_category.py` 爬虫实际未被业务调用。

---

5. 系统模块设计

5.1 编排层模块

5.1.1 ResearchAgent（agent/orchestrator.py）

系统核心类，职责：

方法	职责
`run(user_input)`	同步入口：意图分类 → 路由分发
`stream_run(user_input)`	流式入口：产出事件供 Web UI 消费
`_run_pipeline` / `_stream_pipeline`	执行 9 步串行 Pipeline
`_run_single_tool` / `_stream_single_tool`	执行单工具查询
`_run_qna`	执行通用问答
`_evaluate_pipeline`	Pipeline 决策分支判定
`_get_params_for_stage`	为各阶段构造入参
`_extract_competitor_asins`	从竞品结果提取 ASIN 供评论分析复用
`_generate_summary`	报告生成失败时的降级汇总

关键数据结构：

```
_PIPELINE_STAGES = [
    "market_overview", "trend_analyzer", "category_explorer",
    "competitor_analysis", "review_miner", "keyword_research",
    "profit_calculator", "supplier_search", "product_scorer",
]

_TOOLS: dict[str, Any] = {
    "market_overview": MarketOverviewTool(),
    # ... 共 10 个工具实例
}
```

意图路由逻辑：

```
intent_result = classify_intent(user_input)
if intent_result.intent == "full_pipeline":
    return self._run_pipeline(...)      # 完整调研
elif intent_result.intent == "qna":
    return self._run_qna(...)           # 通用问答
else:
    return self._run_single_tool(...)   # 单工具查询
```

5.1.2 意图分类器（agent/intent_classifier.py）

- 调用 LLM，`temperature=0.1`，将自然语言分类为 11 种意图；
- 返回 `IntentResult`（`intent` / `keyword` / `marketplace` / `additional_params`）；
- 用正则 `\{.*\}` 从 LLM 输出提取 JSON，解析失败则降级为 `qna`；
- 附带 `extract_params` 辅助函数，用正则从文本提取站点（美国/US/UK/德国/DE/日本/JP/加拿大/CA）。

意图枚举：`full_pipeline` / `market_overview` / `trend` / `competitor` / `profit` / `keyword` / `review` / `supplier` / `category` / `score` / `qna`。

5.1.3 状态定义（agent/state.py + models/schemas.py）

`AgentState` 承载会话上下文：

```
class AgentState(BaseModel):
    session_id: str
    user_input: str
    messages: list[Message]
    user: UserState              # 偏好、watchlist、历史分析
    cache: CacheState            # 缓存状态
    current_task: TaskInfo       # 当前任务
    task_history: list[TaskInfo]
    conversation: ConversationState
    results: dict[str, Any]      # 各工具结果
```

5.2 工具层模块（tools/）

5.2.1 统一基类 BaseTool（tools/base.py）

模板方法模式实现，统一封装缓存与异常：

```
class BaseTool(ABC):
    name: str = ""
    description: str = ""

    @abstractmethod
    def execute(self, **kwargs) -> ToolOutput: ...

    def run(self, **kwargs) -> ToolOutput:
        # 1. 记录执行元数据
        # 2. 查询缓存，命中则直接返回（cached=True）
        # 3. 调用 execute()，成功则写入缓存
        # 4. 捕获异常，包装为 ToolOutput(success=False)
```

5.2.2 各工具详细设计

工具	数据源	核心算法	备注

MarketOverviewTool	Amazon 搜索页	`compute_stats` 计算价格/评分/评论数统计；Counter 统计品牌集中度	品牌集中度 = 1 - 品牌多样性
TrendAnalyzerTool	Google Trends	委托 `AmazonSearchCrawler.get_trend` → `GoogleTrendsProvider`	方向判定：后半年均值 > 前半年×1.1 为 up
CategoryExplorerTool	硬编码类目树	递归 `_walk` 构建树，BSR/规模查表估算	不依赖爬虫
CompetitorAnalysisTool	Amazon 商品详情	Listing 质量评分（标题长度+五点+图/视频/A+）	销量基于 BSR 随机估算
ReviewMinerTool	Amazon 评论	好评/差评关键词匹配提取痛点，映射改进建议	抓取失败走 mock 兜底
KeywordResearchTool	Amazon 搜索建议	竞争度按词长估算，CPC 按竞争度映射	建议缺失走预设扩展词兜底
ProfitCalculatorTool	FBA 费率表（硬编码）	佣金 15%（最低 $0.30）+ 配送 + 仓储 + 关税	三档尺寸费率
SupplierSearchTool	1688/Alibaba	委托 `SupplierCrawler`	数据为模拟
ProductScorerTool	各模块评分输入	五维加权求和	权重固定
ReportGeneratorTool	LLM	拼接各模块数据，用 `REPORT_GENERATION_PROMPT` 生成 Markdown	失败降级为 JSON 汇总

5.2.3 利润测算公式（profit_calculator.py）

```
referral_fee    = max(selling_price × 15%, $0.30)
fulfillment     = pick_pack + weight_lb × weight_handling
storage_fee     = (length × width × height / 1728) × monthly_storage
import_duty     = cost_price × tax_rate
total_fees      = referral + fulfillment + storage + shipping + import_d
uty
net_profit      = selling_price − cost_price − total_fees
profit_margin   = net_profit / selling_price × 100%
```

FBA 尺寸档位费率：

档位	pick_pack	weight_handling	monthly_storage
small_standard	$3.50	$0.45	$0.50
large_standard	$4.75	$0.60	$0.75
large_oversize	$8.50	$1.20	$1.50

5.2.4 五维加权评分（product_scorer.py）

维度	权重
市场需求 market_demand	0.30
竞争度 competition	0.25
利润 profitability	0.20
趋势 trend	0.15
供应链 supply_chain	0.10

等级映射：`≥80`→A(go)、`≥65`→B(go)、`≥50`→C(caution)、`<50`→D(abandon)。

5.3 数据层模块（crawlers/）

5.3.1 统一基类 BaseCrawler（crawlers/base.py）

- httpx 客户端，`follow_redirects=True`，随机 User-Agent；
- `request()` / `request_text()`：最多重试 `MAX_RETRIES`（默认 3）次，指数退避（`2**attempt`），随机延迟 `BASE_DELAY + uniform(0,1)`；
- 403/404 直接返回 None（不重试）；
- 超时/网络异常在达到最大重试后返回 None。

5.3.2 各爬虫详细设计

爬虫	抓取目标	解析要点
AmazonSearchCrawler	`/s?k=` 搜索页	选择器 `[data-component-type='s-search-result']`；价格/评分/评论数/品牌/新品标识
	`completion.amazon.com` 建议接口	返回 `data[1]` 建议词列表
	`get_trend`	委托 `GoogleTrendsProvider`
AmazonProductCrawler	`/dp/{asin}` 详情页	标题/价格/评分/评论数/品牌/BSR/五点/图视频 A+
AmazonReviewCrawler	`/product-reviews/{asin}`	`[data-hook='review']` 评论卡片；失败返回 mock
AmazonCategoryCrawler	`/gp/site-directory`	`a[href*='/b/']` 类目链接（备用）
SupplierCrawler	模拟	随机生成 1688/Alibaba 供应商数据
GoogleTrendsProvider	pytrends	月度聚合、方向判定、增长率、峰值月份

站点域名映射：

```
_MARKETPLACES = {"US": "com", "UK": "co.uk", "DE": "de", "JP": "co.jp",
 "CA": "ca"}
```

5.3.3 趋势分析算法（crawlers/trend_provider.py）

1. `pytrends.interest_over_time()` 获取近 12 个月搜索热度；
2. 按月度聚合均值（`_to_monthly`）；
3. 方向判定（`_compute_direction`）：将序列对半分，比较后半段均值与前半段均值，`>1.1` 倍为 up、`<0.9` 倍为 down、否则 stable；
4. 增长率（`_compute_growth`）：`(recent_avg − earlier_avg) / earlier_avg × 100`；
5. 峰值月份（`_compute_peak_months`）：按值降序取前 3 个月。

5.4 展示层模块（frontend/）

模块	职责
`pipeline_progress.py`	生成 Pipeline 步骤条 HTML（9 阶段 + 状态图标 + 旋转 spinner）
`visualizations.py`	Plotly 图表：竞品价格柱状图、趋势折线图、情感饼图、评分分布、评分雷达图
`utils.py`	主题常量、提取最高评分/趋势数据的辅助函数

---

6. 数据库设计

6.1 现状说明（如实）

当前系统无持久化数据库，所有数据均为运行时临时数据，进程退出即丢失。这是需求文档第 6.2 节明确划定的版本边界。系统当前仅使用**进程内内存缓存**（`utils/cache.py`）。

6.2 内存缓存设计

6.2.1 缓存结构

```python
class CacheEntry(BaseModel):
    key: str
    data: dict[str, Any]
    created_at: str        # 创建时间 ISO 格式
    ttl_hours: int = 6     # 存活小时数
    hit_count: int = 0     # 命中次数

class CacheState(BaseModel):
    entries: dict[str, CacheEntry]   # key → entry
```

```
    enabled: bool = True
    default_ttl: int = 6
```

6.2.2 缓存键生成

```
def _make_key(tool_name, params):
    raw = json.dumps(params, sort_keys=True, default=str)
    return f"{tool_name}:{hashlib.md5(raw.encode()).hexdigest()}"
```

缓存键 = 工具名:参数 MD5，保证相同工具+相同参数命中同一缓存。

6.2.3 缓存策略

策略	说明
命中	返回缓存副本，`hit_count += 1`，`cached=True`
过期	超过 TTL（默认 6 小时）自动删除
写入	工具执行成功后自动写入
失效	`invalidate(pattern)` 支持全量或按模式清除

6.3 核心数据模型清单（models/schemas.py）

模型	用途
`Message`	会话消息
`ToolOutput`	工具统一输出契约
`ToolExecRecord`	工具执行记录
`UserPreferences` / `UserState`	用户偏好与状态
`PipelineState`	Pipeline 进度与决策标志
`TaskInfo`	任务信息
`MarketOverview`	市场概览结果
`TrendAnalysis` / `TrendPoint`	趋势分析结果
`CompetitorItem` / `CompetitorAnalysis`	竞品数据
`ProfitDetail`	利润明细
`KeywordItem` / `KeywordResearch`	关键词数据
`ReviewInsight`	评论洞察
`SupplierItem` / `SupplierResult`	供应商数据
`CategoryNode`	类目树节点
`ProductScore`	综合评分

6.4 持久化扩展方案（建议，非当前实现）

若未来引入数据库（需求 P2），建议采用 SQLite（单机）或 PostgreSQL（多租户），表结构设计如下：

```sql
-- 用户表
CREATE TABLE users (
    id            TEXT PRIMARY KEY,
    preferences   JSON,
    created_at    TIMESTAMP
);

-- 调研任务表
CREATE TABLE analysis_tasks (
    task_id       TEXT PRIMARY KEY,
    user_id       TEXT REFERENCES users(id),
    keyword       TEXT,
    marketplace   TEXT,
    intent        TEXT,
    status        TEXT,
    report        TEXT,          -- Markdown 报告
    created_at    TIMESTAMP,
    completed_at  TIMESTAMP
);

-- 工具执行缓存表（持久化缓存，替代内存缓存）
CREATE TABLE tool_cache (
    cache_key     TEXT PRIMARY KEY,
    tool_name     TEXT,
    params_json   TEXT,
    result_json   TEXT,
    created_at    TIMESTAMP,
    expires_at    TIMESTAMP
);

-- 分析结果表（各维度结果）
CREATE TABLE analysis_results (
    id            TEXT PRIMARY KEY,
    task_id       TEXT REFERENCES analysis_tasks(task_id),
    stage_name    TEXT,
    result_json   JSON,
    confidence    TEXT,
    created_at    TIMESTAMP
);
```

---

7. 接口设计

7.1 对外接口（无 REST API）

当前系统**不提供 REST API**，对外仅两种交互方式：

接口　　类型　　入口　　说明

Web UI	Gradio	`http://127.0.0.1:7860`	聊天式交互 + Pipeline 步骤条 + 图表
CLI	命令行	`python main.py`	交互式终端，支持 `/help` `/tools` `/clear` `/cache` `/exit`

7.2 LLM 服务接口（外部调用）

系统调用 OpenAI 兼容 API 或 Anthropic API。

7.2.1 意图分类请求

```
{
  "model": "deepseek-ai/DeepSeek-V3",
  "messages": [
    {"role": "system", "content": "<INTENT_CLASSIFIER_SYSTEM>"},
    {"role": "user", "content": "帮我分析宠物饮水机的市场机会"}
  ],
  "temperature": 0.1
}
```

7.2.2 意图分类响应（LLM 输出文本）

```
{"intent": "full_pipeline", "keyword": "宠物饮水机", "marketplace": "US",
 "additional_params": {}}
```

7.3 工具统一接口（内部）

所有工具遵循 `BaseTool` 统一契约：

```
def execute(self, **kwargs) -> ToolOutput
```

统一输出结构 `ToolOutput`：

```
class ToolOutput(BaseModel):
    success: bool = True
    data: dict[str, Any]        # 业务结果（含 source/confidence/evidenc
e/data_timestamp）
    error: Optional[str] = None
    cached: bool = False
```

数据诚实四字段（每个工具 `data` 中必须包含）：

字段	含义
`source`	数据来源，如“Amazon 搜索结果”、“Google Trends（美国）”
`confidence`	置信度 High / Medium / Low
`evidence`	关键原始证据数据
`data_timestamp`	数据更新时间（ISO 8601 UTC）

7.4 爬虫数据接口（内部）

方法	返回
`AmazonSearchCrawler.search(keyword, marketplace, limit)`	`list[dict]`
`AmazonSearchCrawler.get_suggestions(keyword, marketplace)`	`list[str]`
`AmazonSearchCrawler.get_trend(keyword, marketplace, timeframe)`	`dict | None`
`AmazonProductCrawler.get_product_detail(asin, marketplace)`	`dict | None`
`AmazonReviewCrawler.get_reviews(asin, marketplace, limit)`	`list[dict]`
`AmazonCategoryCrawler.get_category_tree(marketplace)`	`dict | None`
`SupplierCrawler.search(product_name, platform)`	`list[dict]`
`GoogleTrendsProvider.fetch(keyword, marketplace, timeframe)`	`dict | None`

7.5 流式事件协议（Web UI）

`stream_run()` 作为生成器，产出以下事件类型：

事件类型	字段	触发时机
`status`	`text`	意图识别/执行状态提示
`pipeline_start`	`total`, `stage_names`	Pipeline 启动
`progress`	`stage`, `status`, `progress`, `data`, `text`	各阶段状态变化
`abort`	`stage`, `message`, `text`	决策分支提前终止
`result`	`content`	单工具/问答结果
`report`	`content`	完整报告生成

`app.py` 的 `respond()` 消费这些事件，分别驱动 chatbot、Pipeline 步骤条（`render_pipeline_html`）与图表。

---

8. 核心业务流程

8.1 意图分类流程

```
用户输入
   ↓
classify_intent(user_input)
   ↓ LLM 调用 (temperature=0.1)
提取 JSON (正则 \{.*\})
   ↓
   ├─ 解析成功 → IntentResult(intent, keyword, marketplace)
   └─ 解析失败 → IntentResult(intent="qna", keyword="")
```

8.2 完整调研 Pipeline 流程

```
意图 = full_pipeline
   ↓
for stage in [市场概览 → 趋势分析 → 类目穿透 → 竞品分析
              → 评论分析 → 关键词研究 → 利润测算
              → 供应商寻源 → 综合评分]:
    tool.run(**params)
    ├─ 阶段间数据传递：竞品 ASIN → 评论分析
    └─ 决策分支 _evaluate_pipeline:
        ├─ trend_analyzer: direction == "down" → abort
        └─ profit_calculator: profit_margin < 25% → abort
   ↓
report_generator.run(keyword, stage_outputs, language)
   ├─ 成功 → LLM 生成 Markdown 报告
   └─ 失败 → _generate_summary (JSON 简单汇总)
   ↓
返回最终报告
```

8.3 决策分支逻辑

```python
def _evaluate_pipeline(self, stage_name, output, keyword):
    if not output.success:
        return "continue"
    if stage_name == "trend_analyzer":
        if output.data.get("direction") == "down":
            return "abort"
    if stage_name == "profit_calculator":
        if output.data.get("profit_margin", 0) < settings.profit_target
 * 100:
            return "abort"
    return "continue"
```

8.4 单工具查询流程

```
意图 = 具体工具 (如 profit)
   ↓
_TOOLS.get(tool_name)
   ├─ 存在 → tool.run(keyword, marketplace) → JSON 结果
   └─ 不存在 → 降级为通用问答
```

8.5 通用问答流程

```
意图 = qna
   ↓
_run_qna(state):
    取最近 10 条消息 + _QNA_SYSTEM 系统提示
    → llm.invoke() → 返回文本
```

9. 安全设计

9.1 敏感信息保护

保护项	措施
API Key	仅通过环境变量（`.env`）加载，不硬编码、不写入日志
模型配置	`Settings` 统一管理，支持 OpenAI 兼容与 Anthropic 双通道
代理配置	支持 `HTTP_PROXY` / `HTTPS_PROXY` 环境变量，用于合规抓取

关键实现（utils/config.py）：

```
openai_api_key = os.getenv("DEEPSEEK_API_KEY") or os.getenv("OPENAI_API
_KEY")
anthropic_api_key = os.getenv("ANTHROPIC_API_KEY")
```

9.2 访问控制

当前版本**无用户鉴权与登录**（需求文档明确不包含）。系统为本地单机部署，默认绑定 `127.0.0.1`，仅限本机访问。Web UI 若需对外暴露，必须在反向代理层增加认证（如 Nginx Basic Auth 或 OAuth）。

9.3 反爬合规

措施	实现
请求频率控制	`BASE_DELAY`（默认 2s）+ 随机抖动
随机 User-Agent	`_random_ua()` 从 3 个 UA 随机选择
尊重反爬	403 直接停止，不强行重试
失败降级	数据不可获取时返回模拟数据并明确标注（Low 置信度 + note）

需求文档第 16 节明确：系统不得“绕过目标平台的反爬机制进行恶意采集”。

9.4 数据边界

- 系统所有分析结果**仅供参考**，不保证 100% 准确（部分为估算值/代理指标）；
- 报告每个结论必须附带来源、证据、置信度、数据更新时间，防止误导。

---

10. 性能设计

10.1 缓存策略

指标	设计
缓存类型	进程内内存缓存
缓存键	工具名:参数 `MD5`
默认 TTL	6 小时

命中效果　相同参数重复调用直接返回缓存，避免重复爬取/LLM 调用

10.2 爬虫节流

参数	默认值	作用
`CRAWLER_DELAY_SECONDS`	2.0	请求间隔，避免触发反爬
`CRAWLER_MAX_RETRIES`	3	最大重试次数
`CRAWLER_TIMEOUT_SECONDS`	30	单请求超时

重试采用指数退避 `2**attempt` 秒，配合随机抖动。

10.3 超时控制

场景	超时
LLM 调用	`LLM_TIMEOUT_SECONDS`（默认 60s）
爬虫请求	`CRAWLER_TIMEOUT_SECONDS`（默认 30s）
pytrends	15s 固定超时

10.4 并发模型

当前为**单用户交互式**设计，未引入异步并发。Pipeline 串行执行，单次调研耗时取决于 LLM 调用次数与爬取数据量。未来若需多用户并发，需考虑：

- 将同步 `httpx.Client` 改为 `httpx.AsyncClient`；
- 内存缓存改为共享缓存（Redis）；
- Gradio 配置多 worker。

---

11. 异常处理

11.1 异常处理策略总览

异常类型	处理策略
爬虫网络异常	重试 3 次 + 指数退避，最终返回 None
爬虫 403/404	直接返回 None，不重试
数据不可获取	返回空结构 + Low 置信度 + note 说明
LLM 输出无法解析	降级为默认意图（qna）或 JSON 汇总
工具执行异常	捕获后包装为 `ToolOutput(success=False)`
单工具失败	不中断整体 Pipeline（记录错误，继续后续阶段）

11.2 降级机制

场景	降级方案
评论抓取失败	`_mock_reviews()` 返回模拟评论（明确标注）

关键词建议失败	`_fallback_keywords()` 预设扩展词兜底
趋势数据失败	返回 `direction="stable"` + Low 置信度
供应商无结果	返回空列表 + note 说明
报告生成失败	`_generate_summary()` 输出各模块 JSON 汇总

11.3 异常处理代码路径

工具层（tools/base.py）：

```
try:
    output = self.execute(**kwargs)
    output.cached = False
    if output.success:
        cache_util.set(self.name, kwargs, output.data)
except Exception as e:
    output = ToolOutput(success=False, data={}, error=str(e))
```

编排层（main.py）：

```
try:
    result = agent.run(user_input)
except Exception as e:
    result = f"❌ 处理出错：{e}"    # 隐藏内部错误细节
```

12. 日志设计

12.1 现状说明（如实）

当前系统未引入统一的日志框架（无 `logging` 模块使用），仅通过：

- CLI：`rich` 的 `console.print()` / `console.status()` 输出；
- Web：Gradio 的流式事件文本。

这对生产环境运维不利，属于已知缺口。

12.2 建议日志设计（改进方案）

12.2.1 日志级别

级别	用途
`DEBUG`	爬虫请求细节、缓存命中、参数
`INFO`	意图识别结果、Pipeline 阶段开始/完成、工具执行时长
`WARNING`	爬虫重试、数据降级、缓存过期
`ERROR`	工具执行失败、LLM 解析失败、爬虫最终失败

12.2.2 建议日志格式（结构化）

[时间] [级别] [模块] [会话 ID] [事件] 关键字段
例：2026-10-22T10:00:00Z INFO orchestrator sess_abc pipeline_start keyword=宠物饮水机

12.2.3 脱敏要求

日志中**禁止**记录：

- API Key / Token；
- 完整请求头（含 Cookie）；
- 用户敏感信息。

12.2.4 建议实现

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] [%(name)s] %(message)s"
)
```

在 `BaseTool.run()` 中记录执行时长与结果状态，在 `BaseCrawler.request()` 中记录重试次数与最终状态。

13. 部署方案

13.1 本地部署（当前支持）

13.1.1 环境准备

```
# Python 3.10+
cd research_agent
pip install -r requirements.txt
```

13.1.2 配置

```
cp .env.example .env
# 编辑 .env 填入 API Key 与模型名称
```

关键环境变量：

变量	说明	默认值
`OPENAI_API_KEY` / `DEEPSEEK_API_KEY`	OpenAI 兼容 API Key	—
`OPENAI_MODEL_NAME`	模型名称	deepseek-chat
`LLM_BASE_URL` / `DEEPSEEK_API_BASE`	API 地址	—
`ANTHROPIC_API_KEY` / `ANTHROPIC_MODEL_NAME`	Anthropic 配置	—

DEFAULT_MARKETPLACE	默认站点	US
DEFAULT_PROFIT_TARGET	目标利润率	0.25
CRAWLER_DELAY_SECONDS	爬虫延迟	2.0
CACHE_DEFAULT_TTL	缓存 TTL（小时）	6
HTTP_PROXY / HTTPS_PROXY	代理（可选）	—

13.1.3 启动

```
# CLI 模式
python main.py

# Web UI 模式
python app.py    # 浏览器打开 http://127.0.0.1:7860
```

13.2 容器化部署（建议）

当前无 Dockerfile，建议补充：

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 7860
CMD ["python", "app.py"]
```

配合 `docker-compose.yml` 管理环境变量注入与端口映射。

13.3 部署环境依赖

依赖	说明
LLM 服务	需可访问 OpenAI 兼容 API 或 Anthropic API
网络	需可访问 Amazon / Google Trends（或配置代理）
可选代理	访问受限地区需配置 HTTP(S)_PROXY

---

14. 测试方案

14.1 现状说明（如实）

当前项目无自动化测试（无 `tests/` 目录，无 pytest 配置）。功能验证依赖人工运行 CLI/Web 手动测试。这是需优先补齐的缺口。

14.2 建议测试分层

层次	测试对象	工具
单元测试	各工具 execute、缓存、helpers、评分/利润	pytest

	算法	
集成测试	工具 ↔ 爬虫、Pipeline 编排	pytest + mock
接口测试	LLM 意图分类、报告生成	pytest + 录制的 LLM 响应
端到端测试	完整选品调研流程	手动或 playwright

14.3 关键测试用例（对应需求验收标准）

编号	测试项	通过标准
TC-001	意图识别	正确识别 full_pipeline / 单工具 / qna
TC-002	完整调研	输入关键词执行 9 阶段并生成报告
TC-003	趋势下行终止	趋势 direction=down 时 Pipeline 提前 abort
TC-004	利润不达标终止	利润率 < 25% 时提前 abort
TC-005	利润计算	佣金 15%（最低 $0.30）、FBA 三档费率正确
TC-006	综合评分	五维加权求和、等级/推荐动作正确
TC-007	缓存命中	相同参数重复调用命中缓存
TC-008	降级处理	爬虫失败返回空结构 + Low 置信度，不崩溃
TC-009	LLM 解析失败	意图/报告解析失败降级为默认处理
TC-010	单工具失败	单工具失败不中断整体 Pipeline

14.4 建议测试工具与 mock 策略

- 使用 `pytest` + `pytest-mock`；
- 爬虫请求用 `responses` 或 monkeypatch 模拟 HTTP 响应；
- LLM 调用用录制的 fixture（固定输入输出）保证测试确定性；
- 对 `_mock_reviews`、`SupplierCrawler` 等模拟数据，测试其数据结构一致性而非具体数值。

---

15. 风险与扩展

15.1 技术风险

风险	影响	缓解措施
Amazon 反爬封锁	市场/竞品/评论数据不可获取	增加代理池、降低频率、引入官方 API（如 Keepa、Jungle Scout）
Google Trends 不稳定	趋势分析失效	已实现 stable 降级 + Low 置信度；可引入多源趋势数据
LLM 输出不稳定	意图分类/报告格式抖动	正则提取 JSON + 降级默认；使用结构化输出（JSON mode）

数据准确性不足	销量/BSR/供应商为估算或模拟	报告强制标注置信度；引入权威第三方数据源
无持久化	数据丢失、无法追溯历史	引入 SQLite/PostgreSQL（见 6.4）

15.2 业务风险

风险	说明
合规风险	爬取 Amazon/1688 需遵守平台条款与 robots 协议
决策风险	系统仅辅助决策，最终选品责任在用户（需求文档已明确）
数据时效	缓存 TTL 6 小时可能导致数据滞后

15.3 技术债清单

技术债	说明
langgraph 声明未用	移除依赖或迁移到真正的 StateGraph 编排
playwright 声明未用	移除依赖或实现需要浏览器渲染的抓取
模拟数据	供应商、部分评论、销量估算需替换为真实数据源
无日志框架	需引入 logging（见 12.2）
无测试	需补齐自动化测试（见 14.2）
类目树硬编码	需改为动态抓取或配置化

15.4 扩展方向（对应需求 P2）

方向	说明
用户账号体系	引入登录鉴权与多用户数据隔离
持久化存储	保存调研历史、报告、缓存
多租户支持	组织级隔离与权限管理
LangGraph 迁移	将手写 Pipeline 迁移为声明式 StateGraph
更多数据源	接入 Keepa、Helium 10、卖家精灵等
更多站点	扩展 AU、IN、MX 等站点
批量调研	定时任务与批量关键词调研
异步并发	httpx.AsyncClient + 共享缓存，支持多用户

附录

附录 A：与需求文档的追踪关系

需求	技术实现
FR-001 意图识别	`intent_classifier.py`

FR-002 意图路由	`orchestrator.py` 路由逻辑
FR-003 完整 Pipeline	`_run_pipeline` / `_stream_pipeline`
FR-004 市场概览	`MarketOverviewTool`
FR-005 趋势分析	`TrendAnalyzerTool` + `GoogleTrendsProvider`
FR-006 类目穿透	`CategoryExplorerTool`
FR-007 竞品分析	`CompetitorAnalysisTool`
FR-008 评论分析	`ReviewMinerTool`
FR-009 关键词研究	`KeywordResearchTool`
FR-010 利润测算	`ProfitCalculatorTool`
FR-011 供应商寻源	`SupplierSearchTool`
FR-012 综合评分	`ProductScorerTool`
FR-013 报告生成	`ReportGeneratorTool`
FR-014 通用问答	`_run_qna`
FR-015 多站点	`_MARKETPLACES` 映射
FR-016 结果缓存	`utils/cache.py`
FR-017 交互界面	`app.py` + `main.py`
FR-018 配置管理	`utils/config.py` + `.env`

附录 B：配置文件模板（.env.example）

```
# LLM API Configuration (OpenAI / OpenAI-compatible)
OPENAI_API_KEY=sk-your-key-here
OPENAI_MODEL_NAME=gpt-4o
# LLM_BASE_URL=https://api.siliconflow.cn/v1
# ANTHROPIC_API_KEY=sk-ant-your-key-here
# ANTHROPIC_MODEL_NAME=claude-sonnet-4-20250514

# Default marketplace
DEFAULT_MARKETPLACE=US
DEFAULT_CURRENCY=USD
DEFAULT_PROFIT_TARGET=0.25

# Crawler settings
CRAWLER_DELAY_SECONDS=2.0
CRAWLER_MAX_RETRIES=3
CRAWLER_TIMEOUT_SECONDS=30

# Cache TTL in hours
CACHE_DEFAULT_TTL=6
```